Adaptive Replanner in Diffusion Policy via Reinforcement Learning

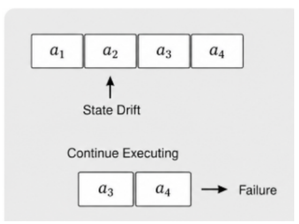
黃冠瑋 [1] 鄭芯薇 [1] 黃浚瑀 [1]

[1]National Yang Ming Chiao Tung University

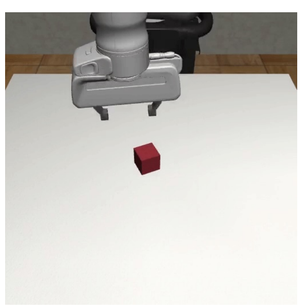
Abstract

This work investigates replanning in Diffusion Policies. Existing methods execute a complete action chunk to reduce inference cost, but state deviations and environmental disturbances can quickly invalidate the remaining actions. We propose a Temporal-Difference Error-based and RL-based adaptive replanner that interrupts unreliable action chunks and generates new actions when necessary. Experiments on Robomimic Lift and Can demonstrate improved both success rates and reward under noisy settings, showing that adaptive replanning enhances the robustness and efficiency of Diffusion Policies.

Motivation



(a) Open-loop Drift Failure



(b) Robomimic Lift

(c) Robomimic Can

Architectural Differences in Replanning Mechanisms

Why is replanning challenging for Diffusion Policies? Let's compare the architectures:

- **MPC (Model Predictive Control):** Relies on an explicit dynamics model to predict future states. It naturally replans whenever the execution deviates from the physics-based forecast.
- **Diffuser (Trajectory-denoising):** Simultaneously generates both actions and future states ($\hat{s}_{t+1}$). It can easily trigger replanning by checking the state deviation $\|\hat{s}_{t+1} - s_{t+1}\|$.
- **Diffusion Policy (D3P / Our Baseline):** Generates an action chunk conditioned *only* on the current state s_t. It has **no learned next-state prediction head**, meaning traditional state-deviation triggers **cannot be applied**.

Two Intuitive Failures when Introducing Replanning to D3P:

- **Per-step Replanning (act_steps $= 1$) breaks continuity:** Forcing the model to replan at every step completely destroys action smoothness. Because Diffusion Policies are inherently **multimodal**, per-step replanning causes the agent to change its mind erratically mid-task (e.g., suddenly switching from a left-side grasp to a right-side grasp), leading to jitter and failure.
- **Naively applying a Replan Trigger encounters OOD states:** If we use a trigger (like TD-error) on a vanilla policy, it will interrupt the execution mid-chunk. However, the vanilla policy was only trained on initial states of a chunk. Being forced to replan from an interrupted, mid-chunk state introduces a severe **distribution gap** (Out-of-Distribution states) that the model cannot handle.

Core Insight: To achieve safe, adaptive replanning, we must close this train-test distribution gap while maintaining action consistency. **Our methodology and RL-based controller successfully bridge this gap.**

Research Objectives

- Analyze the limitations of fixed action chunk execution in Diffusion Policies.
- Develop replanning indicator for Diffusion Policies without explicit state prediction.
- Improve robustness and success rate under noisy and uncertain environments.

TD-triggered Replanner

Preliminaries. D3P (Dynamic Denoising Diffusion Policy) produces an H-step action chunk $\mathbf{c} = (a_0, \ldots, a_{H-1})$ from observation s_t via DDIM denoising, with per-state NFE chosen by a lightweight adaptor A_ψ. We reproduced code in this paper and build our replanner on its actor π_ϕ, critic V_θ, and adaptor A_ψ.

Per-chunk random force-replan training. Vanilla D3P executes a chunk open-loop for H steps, while any deployed replan trigger interrupts mid-chunk — a train/eval distribution gap. During DPPO fine-tuning we close it by truncating each sampled chunk to a uniformly random length:

$$t \sim \mathrm{Uniform}\{1, \ldots, H\}, \qquad \mathrm{exec}(\mathbf{c}) = (a_0, \ldots, a_{t-1}). \tag{1}$$

The next outer step resamples from the resulting mid-chunk state, so all interrupt positions appear in training and PPO credit reaches every chunk position.

TD-triggered replan at inference. We reuse V_θ as a zero-extra-parameter trigger. At every env-step we compute

$$\delta_t = r_t + \gamma\, V_\theta(s_{t+1}) - V_\theta(s_t), \tag{2}$$

and maintain a pooled EMA of its mean / variance ($\alpha = 0.02$) across all parallel envs. After $N = 200$ warmup samples, the trigger fires on downside surprise:

$$\mathrm{trigger}(t) = \mathbb{1}\left[z_t < -\tau\right], \qquad z_t = (\delta_t - \mu)/\sigma, \tag{3}$$

with $\tau = 2$. On fire we discard cached actions and resample via π_ϕ. Forced replan also occurs at chunk exhaustion or episode end, establishing a floor replan rate of $1/H$.

Learned RL-based Replanner

Shared-Trunk Actor-Critic Architecture

A lightweight MLP controller outputs a per-step action $u_t \in \{0 : \text{continue},\ 1 : \text{replan}\}$ from a joint feature vector:

$$\phi_t = \left(s_t,\ a_t^{\text{queued}},\ V_\theta(s_t),\ \delta_{t-1},\ \frac{\text{cursor}_t}{H}\right). \tag{4}$$

To optimize training and save parameters, the network shares a "Trunk MLP" and splits into two heads:

- **Policy Head (Actor):** Outputs the Bernoulli probability for replanning or not.
- **Value Head $V_{\text{ctrl}}(\phi_t)$ (Critic):** Predicts the expected return under the augmented reward.

Note: We cannot reuse the diffusion critic V_θ here because the controller's reward includes an extra computational penalty. Thus, a dedicated V_{ctrl} is required.

Reward Design with Compute Penalty

We augment the environment reward with a Number of Function Evaluations (NFE) penalty to balance task success rate and inference speed:

$$r_t^{\text{ctrl}} = r_t^{\text{env}} - \lambda \cdot \mathrm{NFE}_t. \tag{5}$$

During PPO training, the base diffusion policy π_ϕ, diffusion critic V_θ, and D3P adaptor A_ψ are all **frozen** — the controller is the only trainable module.

Preventing Exploration Collapse

Naive training easily collapses to a trivial "never voluntary replan" strategy because the immediate NFE penalty outweighs the long-term marginal gain. Three structural fixes restore exploration:

- **Entropy Annealing:** Linearly decaying entropy weight from 0.05 to 0.005 over 150 PPO iterations (encourages early exploration, sharpens late policy).
- **Uniform Initialization Bias:** Initializing the head logit to 0 ($p_{\text{replan}} = 0.5$) to prevent the model from biasing toward "continue" at the start.
- **Conservative Cost Scaling:** Setting a small $\lambda = 0.005$ so the compute penalty does not deterministically dominate the success signal.

Forced Replan Floor

As with the heuristic trigger, a replan is automatically forced when $\text{cursor} = H$ or the episode terminates, establishing a theoretical lower-bound replan rate of $1/H$.

Experiment Results

Model estimates

Adaptive replanning significantly enhances D3P baseline under noisy settings. Both TD-triggered and RL-based methods improve performance over the vanilla D3P baseline, with the RL controller achieving the highest success rates and rewards. Furthermore, its performance is comparable to, and in some cases exceeds, that of the Oracle trigger (instant replan when external disturbance occurs). This suggests that optimal replanning is not solely determined by detecting state deviations, but also by learning when intervention is necessary.

Table 1. Evaluation results on robomimic Can and Lift

Task	Method	Success ↑	Reward ↑	Replan rate
4*Lift	Vanilla	0.820	**56.82**	0.25
	TD trigger	**0.855**	56.59	0.255
	RL controller	**0.98**	142.85	0.297
	Oracle	0.965	99.535	0.34
4*Can	Vanilla	0.830	144.29	0.25
	TD trigger	**0.860**	149.47	0.296
	RL controller	**0.87**	152.19	0.257
	Oracle	0.875	152.72	0.362

Conclusions

In conclusion, this work demonstrates that adaptive replanning can be effectively integrated into Diffusion Policies without requiring explicit state prediction. Furthermore, while frequent replanning can improve adaptability, unnecessary replanning may disrupt action consistency. By balancing these two factors, adaptive replanning provides a practical approach for improving the robustness of Diffusion-based control under uncertainty.